01 이상 현상

논리적 설계 단계에서 릴레이션 스키마가 잘못 설계되면 불필요한 데이터 중복이 발생하고, 그로 인해 데이터 관리에 심각한 문제가 생겨난다. 이처럼 관계 DB를 조작할 때 데이터의 중복으로 인해 발생하는 문제를 이상anomaly 현상이라고 한다. [그림 10-1]의 수료하다 릴레이션(기본키는 {학번, 특강번호})은 어떤 학생이 어떤 특강을 수료했는지를 나타내는데, 데이터마다 '학과' 속성의 값이 중복되어 있다. 이 릴레이션에서 C100이라는 특강을 수강한 20220040번 학생의 '학과' 정보에 오류가 있어 '전자'에서 '컴퓨터'로 수정하는 경우 C200, A200, B200 특강의 수료 정보를 나타내는 데이터에서 '학과' 속성의 값을 동시에 수정하지 않으면 이 데이터는 일관성을 잃게 되어 추후 심각한 문제를 유발하는데, 이런 현상이 바로 이상에 해당한다.

수료하다

학번	특강번호	학과	수료여부
20210025	A100	컴퓨터	Pass
20210025	A200	컴퓨터	Pass
20220030	B100	기계	Fail
20230035	C100	건축	Pass
20230035	C200	건축	Pass
20230035	A100	디자인	Pass
20220040	C100	전자	Pass
20220040	C200	전자	Pass
20220040	A200	전자	Pass
20220040	B200	전자	Pass
20240045	C100	컴퓨터	Fail

그림 10-1 이상 현상이 발생하는 수료하다 릴레이션

이상 현상은 데이터를 삽입 · 삭제 · 갱신할 때 발생하므로 릴레이션 스키마를 설계할 때는 발견하기 어렵다. 따라서 설계 시점에 가상의 데이터를 삽입 · 삭제 · 갱신해 보면서 이상 현상을 미리 발견하여 예방하는 것이 바람직하다.

1 이상 현상의 종류

삽입 이상

삽입 이상insertion anomaly은 불필요한 데이터를 함께 삽입하지 않으면 필요한 데이터를 삽입하는 것이 불가능한 현상이다. 예를 들어 [그림 10-1]의 수료하다 릴레이션에 학번이 20240050인 학생이 '기계' 학과라는 사실을 삽입한다고 하자. 이때 수료한 특강 정보 없이 학과 정보만 삽입하는 것이 불가능하다. 왜냐하면 수료하다 릴레이션의 기본키가 {학번, 특강번호}이므로 '특강번호'가 널 값인 경우 삽입이 불가능하기 때문이다. 따라서 원하는 정보('학과')를 삽입하려면 불필요한 정보, 즉 '특강번호'에 임의의 값을 저장해야 한다. 이처럼 불필요한 정보를 강제로 삽입하지 않으면 원하는 정보를 삽입할 수 없는 현상을 삽입 이상이라고 한다.

삭제 이상

삭제 이상deletion anomaly은 어떤 데이터를 삭제하면 유용한 데이터도 함께 삭제되는 현상이다. 예를 들어 [그림 10-1]의 수료하다 릴레이션에서 학번이 20220030인 학생이 'B100' 특강의 수강을 취소해서 이 데이터를 삭제하면, 이 학생이 '기계' 학과라는 정보도 함께 삭제된다. 즉, 유용한 정보를 함께 삭제하지 않으면 어떤 정보를 삭제하는 것이 불가능하고, 어떤 정보를 삭제하면 유용한 정보도 함께 손실되는 현상을 삭제 이상이라고 한다.

갱신 이상

갱신 이상update anomaly은 중복된 데이터 가운데 일부만 갱신되어 데이터의 불일치가 발생하는 현상이다. 예를 들어 [그림 10-1]의 수료하다 릴레이션에서 학번이 20220040인 학생의 학과가 '전자'에서 '컴퓨터'로 갱신된 경우, 학번 20220040에 대한 4개 데이터를 모두 갱신해야 한다. 그렇지 않으면 중복된 데이터의 일부만 갱신되어 정보의 모순이 발생한다. 이처럼 중복된 데이터 가운데 일부만 갱신해서 데이터의 불일치가 발생하는 현상을 갱신 이상이라고 한다.

2 이상 현상 사례

어떤 사원이 어떤 부서에 근무하는지를 나타내는 근무하다 릴레이션(기본키는 {사원번호, 부서번호})에는 사원 정보와 부서 정보가 함께 저장되어 있다. 현재 [그림 10-2]와 같은 데이터가 저장되어 있다고 가정했을 때 다음과 같은 문제가 발생한다.

근무하다

사원번호	부서번호	사원이름	부서이름	주소	휴대폰번호
1001	10	김길동	영업	서울시 강남구	010-1485-3949
1002	20	박영숙	개발	서울시 동작구	010-6283-5542
1002	30	박영숙	기획	서울시 동작구	010-6283-5542
1003	20	이창민	개발	천안시 성정동	010-5756-4857
1004	10	최수민	영업	서울시 강북구	010-3239-4957

그림 10-2 이상 현상이 발생하는 근무하다 릴레이션

- **정보 중복**: 사원번호가 '1002'인 사원이 2개 부서에서 근무하므로, 사원이름, 주소, 휴대폰번호가 중복되어 저장 공간이 낭비된다.
- **삽입 이상**: '연구' 부서를 신설했는데 아직 근무하는 사원이 없다면, 가상의 사원 정보(예 사원번호 '9999', 사원이름 '홍길동')를 같이 입력해야 신설 부서 정보를 삽입할 수 있다.
- **삭제 이상**: 만일 사원번호가 1002인 사원이 퇴사해서 사원 정보를 삭제하면, 30번 부서 정보도 함께 삭제된다.
- **갱신 이상**: 10번 부서의 이름을 '영업'에서 '마케팅'으로 갱신했는데 1001번 사원의 '부서이름'만 변경하고 1004번 사원의 '부서이름'을 변경하지 않으면 데이터의 불일치가 발생하다.

3 이상 현상의 원인과 해결 방법

이상 현상이 발생하는 근본 원인은 릴레이션 스키마를 생성할 때 속성 간의 종속성을 충분히 고려하지 않아 불필요한 데이터가 중복되었기 때문이다. 즉, 속성 간에 존재하는 여러 개의 종속 관계를 하나의 릴레이션으로 표현했기 때문이다.

이상 현상을 해결하려면 릴레이션 스키마를 변환해서 속성 간의 여러 종속 관계를 분해decomposition해야 한다. 즉, 하나의 종속 관계가 하나의 릴레이션으로 표현되도록 릴레이션을 분해해야 하는데, 이러한 과정을 정규화normalization라고 한다. 정규화를 통해 릴레이션 스키마를 더욱 바람직한 방향으로 변환해야 이상 현상을 해결할 수 있다. 구체적인 정규화 방법은 5절에서 자세히 설명한다.

02 함수적 종속과 함수적 종속 다이어그램

함수적 종속Functional Dependency, FD은 정규화의 핵심 개념으로, 관계 데이터 모델을 제안한 코드E. F. Codd 박사가 속성 간의 종속 관계를 규명하기 위해 제안하였다. 함수적 종속은 릴레이션이 갖고 있는 이상 현상을 발견하기 위한 수단으로 사용된다.

어떤 릴레이션 R에서 속성 X의 값이 속성 Y의 값을 결정짓는다고 하자. 즉, 속성 X의 각 값에 대해 속성 Y의 값이 오직 하나만 연관되면 "속성 Y는 속성 X에 함수적으로 종속된다"라고 한다. 이러한 함수적 종속을 X → Y로 표현하고 X를 결정자determinant, Y를 종속자dependent라고 한다. 결정자는 주어진 릴레이션에서 다른 속성을 고유하게 결정하는 하나 이상의 속성을 의미하고, 종속자는 결정자에 의해 값이 결정되는 하나 이상의 속성을 의미한다.

함수적 종속 다이어그램Functional Dependency Diagram, FDD은 한 릴레이션에 속한 속성 간의 복잡한 함수적 종속 관계를 쉽게 이해할 수 있도록 그림으로 표현하는 방법이다. FDD에서는 결정자와 종속자를 모두 직사각형으로 표시하고, 함수적 종속 관계를 화살표로 연결해서 표현한다.

1 함수적 종속 관계 분석 사례(1)

[그림 10-3]과 같은 학생 릴레이션에서 '이름'이 같은 학생이 여러 명 있을 수 있으므로, 학생의 이름을 알아도 그 학생의 학번이나 학년 또는 학과를 알 수 없다. 이름이 학번이나 학년, 학과를 함수적으로 결정하지 않기 때문이다. 즉, 어떤 이름이 오직 하나의 학번이나 학년, 학과하고만 연관된 것이 아니기 때문이다.

학생

학번	이름	학년	학과
20223910	김철수	2	컴퓨터
20233929	이영희	1	전기
20243932	이영희	3	전자
20223934	장영주	2	기계
20233900	김철수	1	디자인

그림 10-3 학생 릴레이션

반면, 모든 학생의 학번이 서로 다르므로 학번을 알면 그 학생의 이름을 알 수 있다. 하나의 학번은 오직 하나의 이름과 연관되어 있으므로 학번이 이름을 함수적으로 결정하기 때문이다. 이름뿐 아니라 학년과 학과 속성도 모두 학번 속성에 함수적으로 종속되어 있다. 따라서 다음과 같이 표현할 수 있다.

```
학번 → 이름
학번 → 학년
학번 → 학과
```

줄여서 다음과 같이 간단히 표현할 수 있다.

```
학번 → (이름, 학년, 학과)
```

또는

```
학번 → 이름 | 학년 | 학과
```

위의 함수적 종속 표현에서 학번은 결정자가 되고 이름, 학년, 학과는 종속자가 된다. 릴레이션 R에서 속성 X가 기본키 또는 후보키이면, R의 모든 속성 Y에 대해 X → Y가 성립한다.

2 함수적 종속 관계 분석 사례(2)

[그림 10-4]는 어떤 사원이 어떤 부서에 어떤 직책으로 소속되는지를 표현하는 소속되다 릴레이션(기본키는 {사원번호, 부서번호})으로, 사원 정보와 부서 정보를 함께 저장하였다. 특히, 이 릴레이션은 한 사원이 2개 이상의 부서에 소속될 수 있고, 각 부서에서 맡은 직책이 다를 수 있음을 표현하고 있다.

소속되다

사원번호	부서번호	부서명	사원명	직책	근무지	휴대폰번호	주소
3214	100	기획팀	김철수	팀장	서울	010-2300-1232	서울시 영등포구
2456	200	개발팀	이영호	프로그래머	대전	010-9932-3234	천안시 서북구
2456	300	연구팀	이영호	설계자	천안	010-9932-3234	천안시 서북구
4602	100	기획팀	박민희	사원	서울	010-3329-0032	서울시 강동구
3722	300	연구팀	김철수	팀장	천안	010-3398-2136	천안시 동남구

그림 10-4 소속되다 릴레이션

소속되다 릴레이션에서 속성 간의 함수적 종속 관계를 분석하면 다음과 같다.

- 사원번호만 알면 사원명과 휴대폰번호, 주소를 알 수 있다. 즉, 사원번호가 사원명과 휴대폰번호, 주소를 함수적으로 결정하므로 함수적 종속 관계는 다음과 같이 표현할 수 있다.

```
사원번호 → (사원명, 휴대폰번호, 주소)
```

- 부서번호를 알면 부서명과 근무지를 알 수 있다. 즉, 부서번호가 부서명과 근무지를 함수적으로 결정하므로 함수적 종속 관계는 다음과 같이 표현할 수 있다.

```
부서번호 → (부서명, 근무지)
```

- 부서명을 알아도 부서번호와 근무지를 알 수 있다. 사람 이름과 달리, 일반적으로 이름이 동일한 부서는 없으므로 부서명이 부서번호와 근무지를 함수적으로 결정한다. 따라서 함수적 종속 관계는 다음과 같이 표현할 수 있다.

```
부서명 → (부서번호, 근무지)
```

- 사원번호와 부서번호를 모두 알아야 직책을 알 수 있다. 즉, {사원번호, 부서번호}가 직책을 함수적으로 결정하므로 함수적 종속 관계는 다음과 같이 표현할 수 있다.

```
{사원번호, 부서번호} → 직책
```

- 동명이인이 존재하므로 사원명을 알아도 사원번호나 부서번호, 주소 등을 알 수 없다. 즉, 사원명을 결정자로 하는 함수적 종속 관계는 존재하지 않는다.
- 소속되다 릴레이션의 함수적 종속 관계를 한꺼번에 표현하면 다음과 같다. {사원번호, 부서번호}는 기본키이므로 직책뿐 아니라 다른 모든 속성의 결정자가 될 수 있다.

```
사원번호 → (사원명, 휴대폰번호, 주소)
부서번호 → (부서명, 근무지)
부서명 → (부서번호, 근무지)
{사원번호, 부서번호} → (직책, 부서명, 사원명, 근무지, 휴대폰번호, 주소)
```

3 함수적 종속 다이어그램 작성 사례(1)

[그림 10-5]와 같은 교과목 릴레이션의 함수적 종속 관계를 분석해서 FDD로 표현하는 과정은 다음과 같다.

교과목

교과목번호	교과목명	학점	학과	담당교수
C123	컴퓨터개론	3	컴퓨터	1102
C234	프로그래밍	3	전자	1200
C257	자료구조	3	컴퓨터	1102
C156	프로그래밍	2	전기	1565
C198	데이터베이스	3	컴퓨터	1422

그림 10-5 교과목 릴레이션

1 함수적 종속 관계 분석

결정자 역할을 하는 속성은 기본키인 '교과목번호'밖에 없으므로 '교과목번호'가 나머지 모든 속성을 함수적으로 결정한다. 이를 함수적 종속 관계로 표현하면 다음과 같다.

교과목번호 → (교과목명, 학점, 학과, 담당교수)

[그림 10-5]에서 확인할 수 있듯이 '프로그래밍'처럼 동일한 교과목명을 갖는 교과목을 서로 다른 학과에서 다른 학점으로 개설하여 서로 다른 교수가 강의할 수 있으므로, 교과목명을 알아도 교과목번호, 학점, 학과, 담당교수를 알 수 없다. 따라서 '교과목명'은 어떤 다른 속성의 결정자가 될 수 없다. '학점', '학과', '담당교수' 속성도 마찬가지다.

2 FDD 표현

'교과목번호'가 다른 모든 속성의 결정자 역할을 하는 함수적 종속 관계를 FDD로 표현하면 [그림 10-6]과 같다.

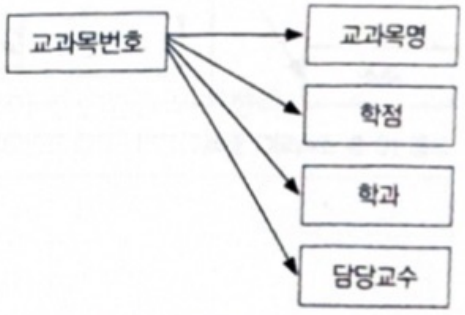


그림 10-6 교과목 릴레이션의 FDD

4 함수적 종속 다이어그램 작성 사례(2)

[그림 10-4]의 소속되다 릴레이션의 함수족 종속 관계를 FDD로 표현하면, [그림 10-7]과 같이 각각 표현하거나 [그림 10-8] 또는 [그림 10-9]와 같이 하나의 다이어그램으로 표현할 수 있다. 이 중 기본키를 중심으로 [그림 10-9]와 같이 표현하는 것이 일반적이다. {사원번호, 부서번호}는 기본키이므로 '직책'뿐 아니라 나머지 모든 속성의 결정자가 될 수 있지만, 기본키가 복합키인 경우 간단히 완전 함수적 종속 관계만 표현하는 것이 일반적이다. 완전 함수적 종속에 대해서는 바로 뒤에 이어서 자세히 설명한다.

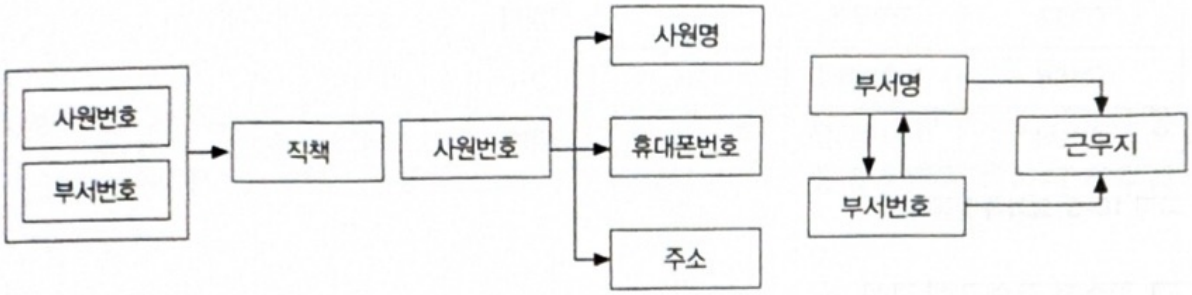


그림 10-7 소속되다 릴레이션의 FDD 표현(1)

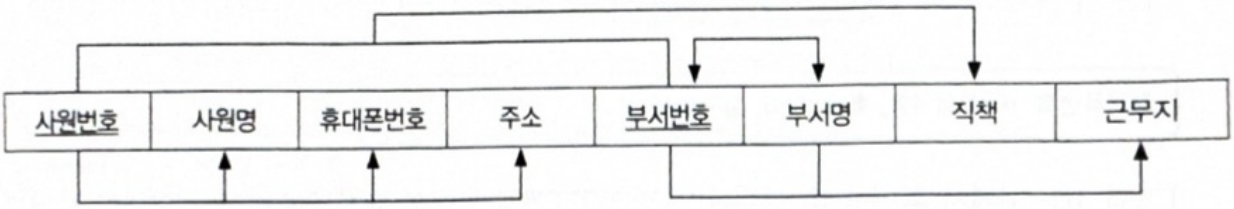


그림 10-8 소속되다 릴레이션의 FDD 표현(2)

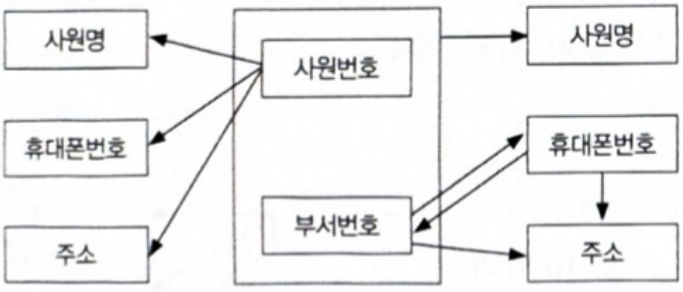


그림 10-9 소속되다 릴레이션의 FDD 표현(3)

03 완전 함수적 종속과 부분 함수적 종속

결정자가 복합키인 경우, 종속자를 완전 함수적 종속 또는 부분 함수적 종속으로 구분할 수 있다.

1 완전 함수적 종속

완전 함수적 종속full functional dependency이란 다음 조건이 성립함을 의미한다.

> 복합키 X에 대해 X → Y가 성립할 때,
> X' ⊂ X이면서 X' → Y를 만족하는 속성 X'이 존재하지 않으면,
> 속성 Y는 복합키 X에 완전 함수적 종속이 된다.

즉, 결정자인 복합키에 포함되는 속성 가운데 단독으로 결정자 역할을 하는 속성이 없으면, 종속자가 결정자에 완전 함수적 종속이 된다. 완전 함수적 종속을 FDD로 표현하면 [그림 10-10]과 같다. 이 FDD는 복합키인 {A, B}가 C의 결정자이고, A나 B만으로는 C를 결정할 수 없음을 나타낸다.

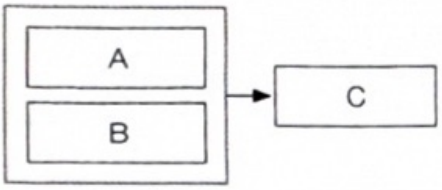


그림 10-10 완전 함수적 종속의 FDD 표현

2 부분 함수적 종속

부분 함수적 종속partial functional dependency이란 다음 조건이 성립함을 의미한다.

> 복합키 X에 대해 X → Y가 성립할 때,
> X' ⊂ X이면서 X' → Y를 만족하는 속성 X'이 존재하면,
> 속성 Y는 복합키 X에 부분 함수적 종속이 된다.

즉, 결정자인 복합키에 포함되는 속성 가운데 단독으로 결정자 역할을 하는 속성이 있으면, 종속자가 결정자에 부분 함수적 종속이 된다. 부분 함수적 종속을 FDD로 표현하면 [그림 10-11]과 같다. 이 FDD는 복합키인 {A, B}가 C의 결정자이지만, B만으로도 C가 결정됨을 나타낸다.

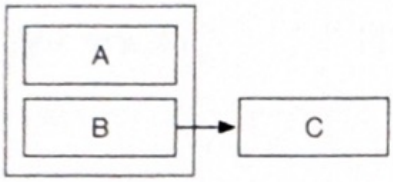


그림 10-11 부분 함수적 종속의 FDD 표현

3 완전 및 부분 함수적 종속 관계 분석 사례(1)

어떤 회원이 어떤 다른 회원의 추천을 받아서 어떤 모임에 가입했는지를 나타내는 '가입하다(회원번호, 모임번호, 회원명, 휴대폰번호, 가입일자, 추천회원번호(FK))' 릴레이션의 함수적 종속 관계는 다음과 같고, FDD로 표현하면 [그림 10-12]와 같다.

회원번호 → (회원명, 휴대폰번호)
{회원번호, 모임번호} → (가입일자, 추천회원번호)

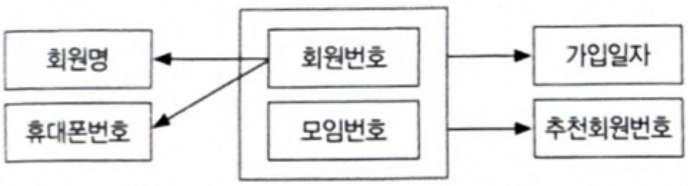


그림 10-12 가입하다 릴레이션의 FDD 표현

기본키인 복합키 X = {회원번호, 모임번호}에 대해 각 속성의 완전 및 부분 함수적 종속 관계를 분석하면 다음과 같다.

- 가입일자, 추천회원번호: X에 완전 함수적 종속이다.
- 회원명, 휴대폰번호: X에 부분 함수적 종속이다.

4 완전 및 부분 함수적 종속 관계 분석 사례(2)

어떤 학생이 어떤 특강을 수료했는지 여부를 나타내는 '수료하다(학번, 특강번호, 학과, 수료여부)' 릴레이션의 함수적 종속 관계는 다음과 같고, FDD로 표현하면 [그림 10-13]과 같다.

{학번, 특강번호} → 수료여부
학번 → 학과

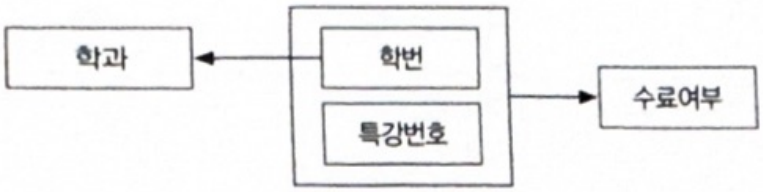


그림 10-13 수료하다 릴레이션의 FDD 표현

기본키인 복합키 X = {학번, 특강번호}에 대해 각 속성의 완전 및 부분 함수적 종속 관계를 분석하면 다음과 같다.

- 수료여부: X에 완전 함수적 종속이다.
- 학과: X에 부분 함수적 종속이다.

04 이행적 함수적 종속

이행적 함수적 종속transitive functional dependency이란 다음 조건이 성립함을 의미하며, 이를 FDD로 표현하면 [그림 10-14]와 같다.

> 릴레이션 R에 함수적 종속 A → C가 존재하고,
> 함수적 종속 A → B ∧ B → C가 성립할 때,
> 속성 C는 속성 A에 이행적 함수적 종속이 된다.

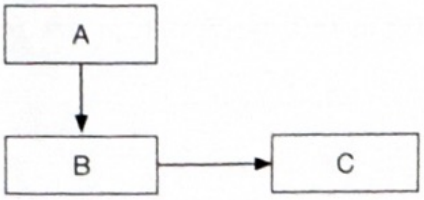


그림 10-14 이행적 함수적 종속의 FDD 표현

예를 들면 '학생(학번, 교과목번호, 성적, 학과명, 학과전화번호)' 릴레이션의 함수적 종속 관계와 FDD 표현은 [그림 10-15]와 같다.

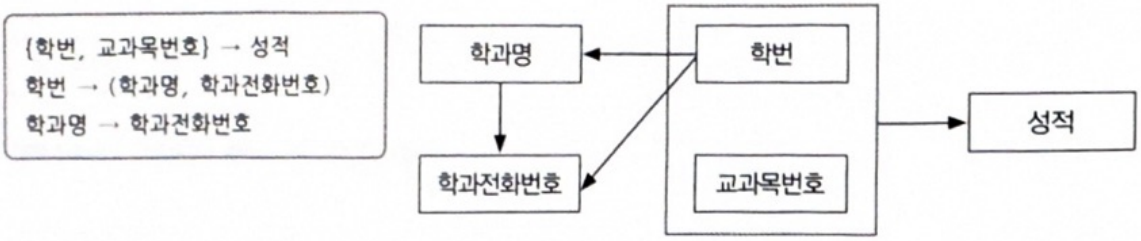


그림 10-15 학생 릴레이션의 FDD 표현

[그림 10-15]에서 학과전화번호 속성은 이행적(또는 추이적)으로 학번 속성에 종속하기 위한 다음 조건이 성립하므로 이행적 함수적 종속이 존재한다.

> 학번 → 학과명 ∧ 학과명 → 학과전화번호

05 정규화

1 정규화 개념

정규화normalization의 기본 원칙은 '독립적인 관계는 서로 다른 릴레이션으로 표현하자'는 것으로, 다음과 같이 여러 가지 관점에서 정규화를 정의할 수 있다.

- 정규화는 속성 간의 종속성을 분석하여 하나의 릴레이션에는 하나의 종속성만 갖도록 릴레이션을 분해하는 과정이다.
- 정규화는 이상 현상의 발생 요인인 데이터 중복을 최소화하기 위해서 일종의 제약조건을 기초로 하여 릴레이션을 분해하는 과정이다.
- 정규화는 스키마 변환을 통해서 일련의 제약조건을 만족하는 릴레이션을 만드는 과정이다.
- 정규화는 불필요한 데이터 중복을 제거해서 논리적 데이터 모델을 단순화하는 과정이다.

릴레이션을 제대로 정규화하면 다음과 같은 효과를 기대할 수 있다.

- 중복된 데이터가 제거되어 이상 현상이 감소한다.
- 데이터의 일관성을 유지할 수 있다.
- 데이터 모델이 단순해진다.
- 데이터 무결성을 유지하는 데 필요한 프로그램 코드량이 감소한다.

2 정규화 원칙

관계형 DB의 논리적 구조는 정규화에 의한 스키마 변환을 통해서 얻게 되는 최종 스키마를 의미한다. 스키마를 변환할 때는 다음과 같은 정규화 원칙에 기초해서 진행한다.

■ **데이터 중복을 최소화**minimal data redundancy**한다.**

데이터 중복으로 인해 여러 가지 이상 현상이 발생하므로, 전체적으로 데이터 중복을 감소시키고 최소한의 중복만 허용해야 한다.

■ **정보의 무손실이 보장되어야 한다.**

같은 의미의 정보를 유지하면서 더 바람직한 구조로 변환해야 한다. 즉, 릴레이션 분해로 인해 손실되는 정보가 없어야 한다. 따라서 필요한 경우 분해된 릴레이션들로부터 원래의 릴레이션을 다시 구할 수 있어야 한다. 만약 릴레이션 분해를 잘못하면 분해된 두 릴레이션으로부터 얻을 수 있는 정보가 원래의 릴레이션이 제공하던 정보보다 적거나 많을 수 있다. 분해된 두 릴레이션으로부터 얻을 수 있는 정보는 원래의 릴레이션이 갖고 있던 정보와 정확히 일치해야 한다.

■ **다음과 같은 분리 원칙principles of separation에 따라 분해한다.**

- 독립적인 관련성은 별도의 릴레이션으로 표현한다.
- 릴레이션 각각에 대해 독립적인 조작이 가능해야 한다.

정규화를 통해 릴레이션을 더 바람직한 형태로 변환할 수 있지만, 릴레이션 분해로 인해 많은 조인join 연산이 발생하여 질의에 대한 응답시간이 느려질 수 있다는 문제가 있다. 조인 연산이란 두 릴레이션과 관련된 투플을 하나의 투플로 결합하는 연산을 의미한다. 따라서 성능을 고려해서 역정규화를 고려해야 할 때도 있는데, 이에 대해서는 7절에서 자세히 설명한다.

한편, 릴레이션을 정규화하지 않으면 다음과 같은 문제가 발생할 수 있다.

- 데이터 중복으로 인해 이상 현상이 발생한다.
- 특정 정보를 표현하는 것이 불가능할 수 있다.
- 널null 값을 많이 포함하게 된다.
- 저장된 정보가 부정확할 수 있다.

3 정규형 개념

정규형normal form은 어떤 제약조건을 만족하는 릴레이션을 의미한다. 정규형에는 제1 정규형First Normal Form, 1NF~제5 정규형Fifth Normal Form, 5NF, 보이스/코드 정규형Boyce/Codd Normal Form, BCNF의 여섯 가지가 있다. 정규화, 즉 스키마 변환($S \rightarrow S'$)을 통해서 점차 높은 수준의 정규형을 만들어가게 되는데, 정규형의 차수가 높아질수록 만족시켜야 하는 제약조건도

더 많아진다. 한편, 관계형 DB는 제1 정규형만 취급한다. 즉, 최소한 제1 정규형의 제약조건을 만족하는 릴레이션만 관계형 DB에 포함될 수 있다.

1NF~5NF는 [그림 10-16]과 같은 포함 관계가 성립한다. 따라서 가장 높은 수준인 제5 정규형은 만족해야 하는 제약조건이 가장 많다. 또한 정규화 정도가 높을수록 릴레이션이 더 많이 분해되어 중복된 데이터가 감소하는 반면, 정규화 정도가 낮을수록 하나의 릴레이션에 많은 정보가 포함되어 있어 조인 연산이 불필요하므로 응답시간이 단축된다는 장점이 있다. 결국 '데이터의 불필요한 중복 감소'와 '응답시간 단축'은 상호 상반되는 목표라고 할 수 있다.

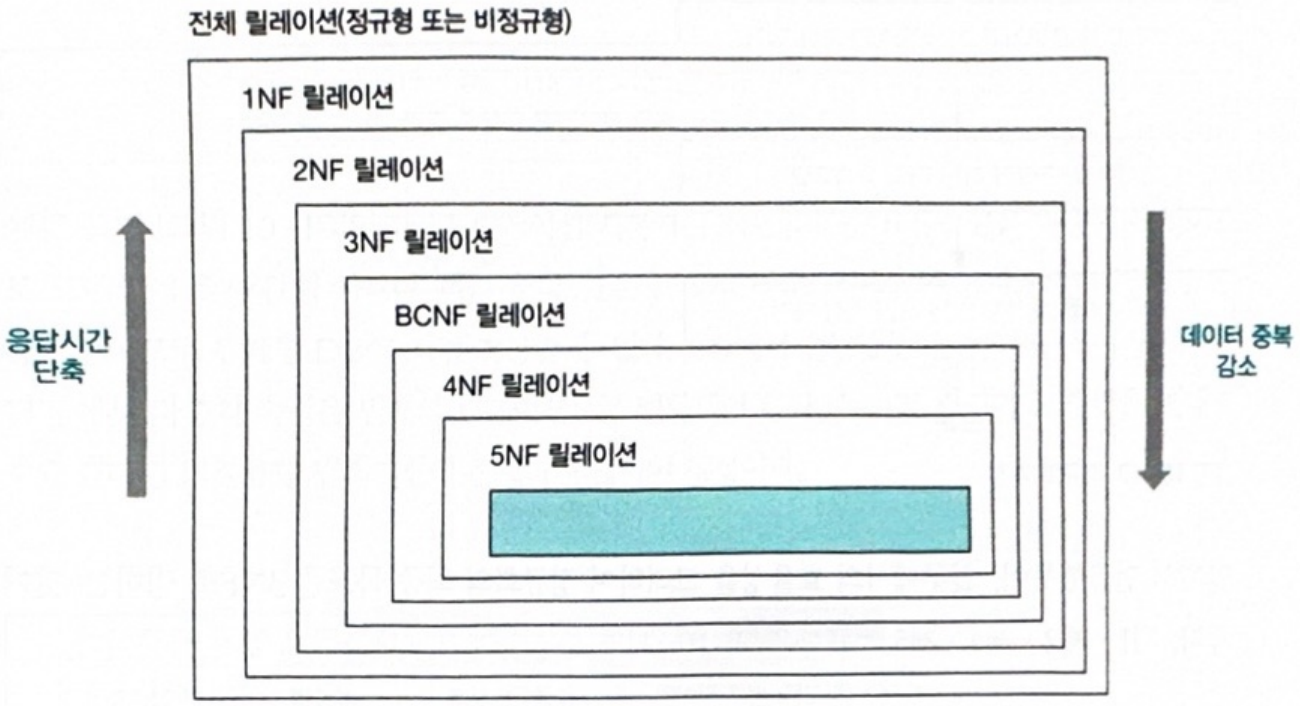


그림 10-16 정규형 간의 포함 관계

4 정규화 과정

정규화 과정은 [그림 10-17]과 같이 요약되며, 단계별로 정규형이 만족해야 하는 제약조건과 정규화 방법에 차이가 있다. 정규화 과정은 이론적으로 5NF까지 있지만 실무에서의 효율성을 고려하여 3NF까지만 진행하면 된다. 따라서 이 책에서도 3NF까지만 다룬다. 릴레이션의 정규화는 실제 데이터 값이 아닌 개념적 측면에서 수행해야 하며, 실제 정규화 과정을 진행할 때 모든 단계를 순서대로 밟아 진행하는 것은 아니다. 즉, 릴레이션의 특성에 따라 어떤 정규형은 건너뛸 수도 있다.

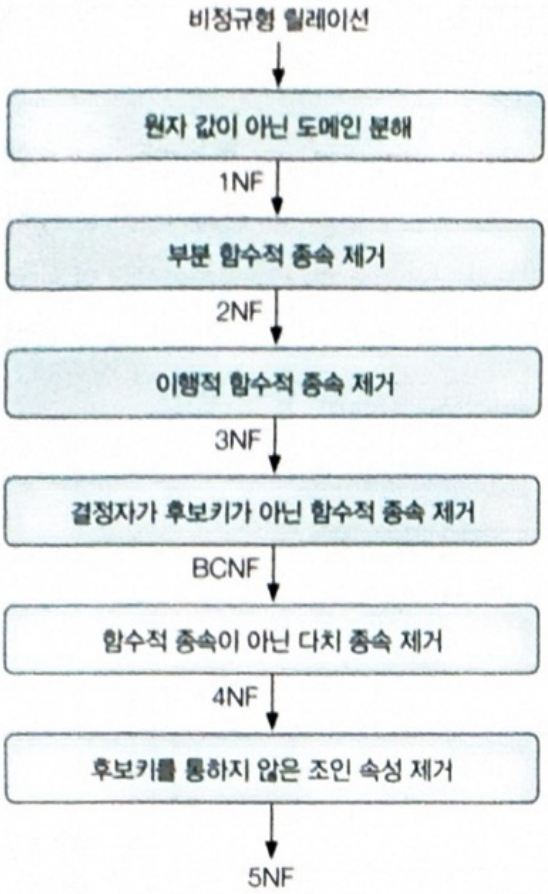


그림 10-17 정규화 과정

앞에서 언급했듯이, 실무에서의 효율성을 고려하여 정규화의 최종 목표를 3NF로 정하는 것이 좋다. 제1 · 제2 · 제4 · 제5 정규형은 [표 10-1]과 같은 문제점을 내포하고 있기 때문이다.

표 10-1 제1·제2·제4·제5 정규형의 문제점

정규형	문제점
1NF, 2NF	• 3NF와 비교하면 불필요한 중복이 많다. • 불필요한 중복으로 인해 기억공간의 낭비가 크다.
4NF, 5NF	• 3NF와 비교하면 릴레이션 수가 지나치게 많아질 수 있다. 즉, 릴레이션이 지나치게 세분화되어 조인(join) 연산이 증가하므로 응답시간이 늦어진다. • 함수적 종속이 아닌 다치 종속(multi-valued dependency)과 조인 종속(join dependency)이라는 복잡한 형태의 종속성까지 고려해야 한다.

06 제1 · 제2 · 제3 정규형

1 제1 정규형

관계형 DB는 최소한 제1 정규형First Normal Form, 1NF을 만족하는 릴레이션만 포함할 수 있는데, 1NF가 만족해야 하는 제약조건은 다음과 같다.

> **제1 정규형(1NF)은 어떤 릴레이션 R에 속한**
> **모든 속성의 도메인이 원자 값을 가져야 한다.**

예를 들면 [그림 10-18]의 학생1 릴레이션(기본키는 '학번')의 '동아리' 속성은 다중치 속성이므로 도메인이 원자 값이 아니다. 따라서 학생1은 1NF의 제약조건을 만족하지 않는 비정규형 릴레이션이므로 관계형 DB에 직접 저장할 수 없다. 8장에서 설명했듯이, 논리적 스키마 모델링 단계에서 다중치 속성은 별개의 릴레이션으로 변환해야 한다. 논리적 스키마 모델링을 제대로 수행했다면 비정규형 릴레이션이 존재하지 않아야 정상이다.

학생1

학번	이름	학과	동아리
20223426	박하늘	컴퓨터공학과	(영어회화반, 검도부)
20235914	이효리	기계공학과	(수화반, 합창반)
20248540	김현민	전자공학과	미술반
20230264	이정우	전기공학과	검도부
20246123	김명길	신소재공학과	축구부

그림 10-18 다중치 속성을 포함하는 비정규형 릴레이션 사례

하지만 논리적 스키마 모델링을 제대로 수행하지 않아서 [그림 10-18]과 같은 비정규형 릴레이션이 존재한다면, 모든 속성이 원자 값만 갖는 1NF로 변환해야 한다. 즉, 동아리 속성에 있는 복수 개의 값 각각에 대해 투플을 하나씩 생성해서 [그림 10-19]의 학생2 릴레이션(기본키는 {학번, 동아리})으로 변환해야 관계형 DB에 저장할 수 있다.

학생2

학번	이름	학과	동아리
20223426	박하늘	컴퓨터공학과	영어회화반
20223426	박하늘	컴퓨터공학과	검도부
20235914	이효리	기계공학과	수화반
20235914	이효리	기계공학과	합창반
20248540	김현민	전자공학과	미술반
20230264	이정우	전기공학과	검도부
20246123	김명길	신소재공학과	축구부

그림 10-19 제1 정규형을 만족시키기 위해 변환한 릴레이션 사례

단, 학생2 릴레이션의 기본키가 {학번, 동아리}인 점에 유의해야 한다. 만약 기본키를 원래대로 '학번'으로 정의하면, 2번째와 4번째 데이터는 개체 무결성을 위반하므로 삽입할 수 없게 된다. 최소한 1NF로 정규화된 릴레이션은 관계형 DB에 저장할 수 있지만, 1NF도 부분 함수적 종속으로 인해 이상 현상이 발생할 수 있다.

2 제2 정규형

1NF에서 이상 현상이 발생하는 원인은 부분 함수적 종속 관계가 있는 속성이 존재하기 때문이다. 1NF의 이상 현상을 해결하려면 릴레이션을 분해하여 부분 함수적 종속을 제거함으로써 제2 정규형Second Normal Form, 2NF을 만족시켜야 한다. 2NF가 만족해야 하는 제약조건은 다음과 같다.

> 제2 정규형(2NF)은 어떤 릴레이션 R이 **1NF이면서**
> **기본키가 아닌 모든 속성이 기본키에 완전 함수적 종속**이어야 한다.

1NF는 만족하지만 2NF를 만족하지 않는 릴레이션 사례를 살펴보자. '수강지도하다(학번, 교과목번호, 지도교수번호, 학과, 성적)' 릴레이션의 함수적 종속 관계와 FDD 표현은 [그림 10-20]과 같다.

{학번, 교과목번호} → 성적
학번 → (지도교수번호, 학과)
지도교수번호 → 학과

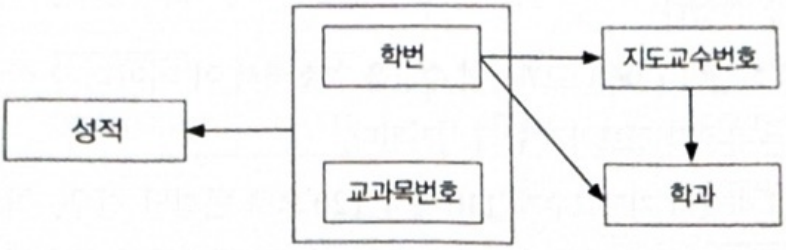


그림 10-20 수강지도하다 릴레이션의 FDD

릴레이션 정규화는 실제 데이터 값이 아닌 개념적 측면에서 수행해야 하지만, 이해를 돕기 위해 수강지도하다 릴레이션에 [그림 10-21]과 같은 데이터가 저장되었다고 가정한다.

수강지도하다

학번	교과목번호	지도교수번호	학과	성적
20223426	C325	110	컴퓨터	B
20223426	E517	110	컴퓨터	B
20235914	C960	235	기계	F
20248540	C546	332	디자인	A
20248540	C235	332	디자인	B
20248540	C869	332	디자인	C
20230264	C546	110	컴퓨터	B
20230264	C235	110	컴퓨터	B
20230264	C869	110	컴퓨터	C
20230264	E517	110	컴퓨터	A

그림 10-21 수강지도하다 릴레이션

수강지도하다 릴레이션의 이상 현상을 분석하여 릴레이션을 분해하는 과정은 다음과 같다.

1 1NF인 수강지도하다 릴레이션의 이상 현상 분석

- **삽입 이상**: 학번이 '20242556'인 학생의 지도교수가 '332'라는 사실을 삽입하려 할 때, 어떤 교과목을 수강하지 않는 한 삽입하는 것이 불가능하다. 즉, 임의의 교과목번호를 삽입하지 않으면 지도교수 정보를 삽입할 수 없다.
- **삭제 이상**: 학번이 '20235914'인 학생이 'C960' 교과목의 수강을 취소해서 이 데이터가 삭제되는 경우, 지도교수가 '235'라는 유용한 정보까지 함께 삭제된다.
- **갱신 이상**: 학번이 '20230264'인 학생의 지도교수가 '110'에서 '120'으로 변경된 경우, 학번이 '20230264'인 4개 데이터의 '지도교수번호' 속성 값을 모두 '120'으로 갱신하지 않으면 데이터 불일치가 발생한다.

2 수강지도하다 릴레이션의 이상 현상 원인

'지도교수번호'와 '학과' 속성이 기본키인 {학번, 교과목번호}에 부분 함수적 종속 관계가 존재하기 때문에 1NF임에도 불구하고 이상 현상이 발생한다.

3 수강지도하다 릴레이션의 이상 현상 해결: 릴레이션 분해

부분 함수적 종속으로 인한 이상 현상을 해결하려면 릴레이션을 분해해서 부분 함수적 종속을 제거해야 한다. 즉, 수강지도하다 릴레이션에서 부분 함수적 종속 관계를 맺는 속성들을 분리해서 다음과 같이 '지도받다'와 '수강하다'라는 2개의 릴레이션으로 분해함으로써 부분 함수적 종속을 제거할 수 있다.

지도받다(<u>학번</u>, 지도교수번호, 학과)
수강하다(<u>학번(FK), 교과목번호</u>, 성적)

[그림 10-22]는 분해된 2개의 릴레이션을 나타낸 것으로, 지도받다 릴레이션은 부분 함수적 종속이 제거되어 모든 속성이 기본키에 완전 함수적 종속이므로 2NF의 제약조건을 만족한다. 하지만 수강하다 릴레이션은 기본키가 복합키이므로 부분 함수적 종속으로 인해 이상 현상이 발생하는지 다시 분석해야 하는데, '성적' 속성이 기본키인 {학번, 교과목번호}에 완전 함수적 종속이므로 2NF의 제약조건을 만족한다.

지도받다

학번	지도교수번호	학과
20223426	110	컴퓨터
20235914	235	기계
20248540	332	디자인
20230264	110	컴퓨터

수강하다

학번	교과목번호	성적
20223426	C325	B
20223426	E517	B
20235914	C960	F
20248540	C546	A
20248540	C235	B
20248540	C869	C
20230264	C546	B
20230264	C235	B
20230264	C869	C
20230264	E517	A

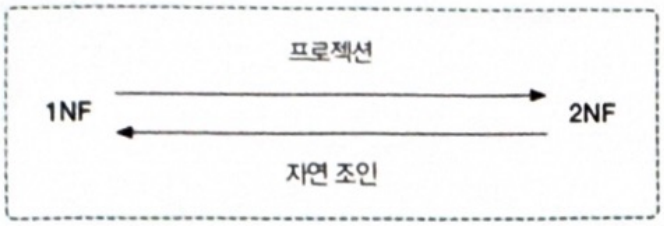
그림 10-22 수강지도하다 릴레이션의 부분 함수적 종속 제거를 위해 분해된 2개의 릴레이션

만약 수강지도하다 릴레이션을 '지도받다(학번, 지도교수번호)'와 '수강하다(학번(FK), 교과목번호, 성적, 학과)'로 분해하면, '학번 → (지도교수번호, 학과)'와 '지도교수번호 → 학과'라는 함수적 종속을 잃어버리게 된다. 즉, 어떤 교수가 어떤 학과 소속인지를 나타내는 데이터를 삽입할 수 없으므로 유의해야 한다. 따라서 반드시 다음과 같은 원칙을 지켜서 [그림 10-22]와 같이 분해해야 한다.

> 하나의 릴레이션에 {A, B} → C라는 완전 함수적 종속과
> A → (D, E)라는 부분 함수적 종속이 함께 존재할 때,
> {A, B, C}와 {A, D, E}로 분해해서 부분 함수적 종속을 제거한다.

부분 함수적 종속은 결정자가 복합키일 때만 발생할 수 있다. 따라서 릴레이션의 기본키가 단일 속성인 경우, 릴레이션이 1NF를 만족하면 2NF도 만족하게 된다. 즉, 기본키가 복합키인 경우에만 2NF의 제약조건을 만족하는지 분석하면 된다.

1NF이면서 2NF가 아닌 릴레이션은 프로젝션projection을 통해서 의미상 동등한 2개의 2NF 릴레이션으로 분해할 수 있다. 반대로, 분해된 2개의 2NF 릴레이션을 자연 조인natural join하면 아무런 정보 손실 없이 원래의 릴레이션으로 되돌릴 수 있다. 이처럼 아무런 정보 손실 없이 한 릴레이션을 분해하는 것을 무손실 분해nonloss decomposition라고 한다.



이처럼 무손실 분해를 하면 원래의 릴레이션에 표현할 수 있는 데이터를 분해된 릴레이션들에도 표현할 수 있다. 그러나 그 역이 성립하는 것은 아니다. 예를 들면, 앞의 사례에서 학번이 '20242556'인 학생의 지도교수가 '332'라는 사실을 1NF인 수강지도하다 릴레이션([그림 10-21])에서는 표현할 수 없었으나, 분해된 지도받다 릴레이션([그림 10-22])에서는 표현할 수 있다. 따라서 2NF로 정규화된 릴레이션이 실세계의 데이터를 더 자연스럽게 표현하는 구조라고 볼 수 있다. 하지만 2NF도 이행적 함수적 종속으로 인해 이상 현상이 발생할 수 있다.

3 제3 정규형

2NF에서 이상 현상이 발생하는 원인은 이행적 함수적 종속 관계인 속성이 존재하기 때문이다. 2NF의 이상 현상을 해결하려면 릴레이션을 분해해서 이행적 함수적 종속을 제거함으로써 제3 정규형Third Normal Form, 3NF을 만족시켜야 한다. 3NF가 만족해야 하는 제약조건은 다음과 같다.

> 제3 정규형(3NF)은 어떤 릴레이션 R이 **2NF이면서,**
> **기본키가 아닌 모든 속성이 기본키에 이행적 함수적 종속이 아니어야** 한다.

2NF는 만족하지만 3NF를 만족하지 않는 사례로 '지도받다(학번, 지도교수번호, 학과)' 릴레이션의 함수적 종속 관계와 FDD 표현은 [그림 10-23]과 같다.

학번 → (지도교수번호, 학과)
지도교수번호 → 학과

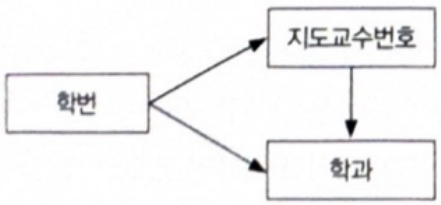


그림 10-23 지도받다 릴레이션의 FDD

지도받다 릴레이션에 [그림 10-22]와 같은 데이터가 저장되었다고 가정하고, 이상 현상을 분석하여 릴레이션을 분해하는 과정은 다음과 같다.

1 2NF인 지도받다 릴레이션의 이상 현상 분석

- **삽입 이상**: 지도교수번호가 '120'인 교수의 학과가 '전자'라는 사실을 삽입하려 할 때, 학번이 기본키이므로 지도 학생이 없으면 삽입하는 것이 불가능하다. 즉, 임의의 학번을 삽입하지 않으면 지도교수의 학과 정보를 삽입할 수 없다.
- **삭제 이상**: 학번이 '20248540'인 학생의 정보를 삭제하면, 지도교수 '332'의 학과가 '디자인'이라는 유용한 정보까지 함께 삭제된다.
- **갱신 이상**: 지도교수 '110'의 학과가 '컴퓨터'에서 '전자'로 변경되는 경우, 학번이 '20223426'과 '20230264'인 2개 투플의 학과를 모두 '전자'로 갱신하지 않으면 데이터 불일치가 발생해서 정보의 일관성을 잃게 된다.

2 지도받다 릴레이션의 이상 현상 원인

2NF인데도 여러 가지 이상 현상이 발생하는 이유는 [그림 10-23]에서 알 수 있듯이 '학번', '지도교수번호', '학과'라는 3개의 속성 사이에 이행적 함수적 종속 관계가 존재하기 때문이다. 즉, 지도받다 릴레이션은 학생과 지도교수 간의 관계만 표현해야 하는데, 지도교수의 학과 정보까지 함께 표현했기 때문에 이상 현상이 발생했다.

3 지도받다 릴레이션의 이상 현상 해결: 릴레이션 분해

이행적 함수적 종속으로 인한 이상 현상을 해결하려면 릴레이션을 분리하여 다음과 같이 '학생지도'와 '교수소속'이라는 2개의 릴레이션으로 분해함으로써 이행적 함수적 종속을 제거해야 한다. 이때 '지도교수번호' 속성을 교수소속 릴레이션의 기본키를 참조하는 외래키로 지정한다는 점에 유의한다.

학생지도(<u>학번</u>, 지도교수번호(FK))
교수소속(<u>교수번호</u>, 학과)

[그림 10-24]는 지도받다 릴레이션이 분해된 2개의 릴레이션을 나타낸 것으로, 이처럼 이행적 함수적 종속이 제거된 정규형을 제3 정규형3NF이라 한다.

학생지도

학번	지도교수번호
20223426	110
20235914	235
20248540	332
20230264	110

교수소속

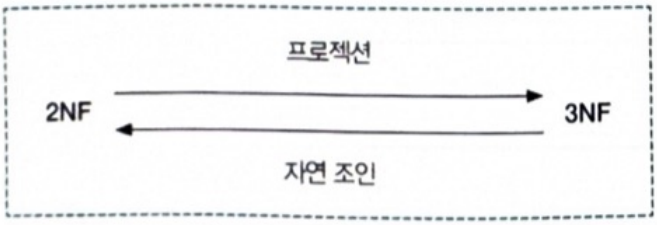
교수번호	학과
110	컴퓨터
235	기계
332	디자인

그림 10-24 지도받다 릴레이션의 이행적 함수적 종속 제거를 위해 분해된 2개의 릴레이션

만약 지도받다 릴레이션을 '학생지도(학번, 지도교수번호)'와 '학생소속(학번(FK), 학과)'로 분해하면, '지도교수번호 → 학과'라는 함수적 종속을 잃어버리게 된다. 즉, 어떤 교수가 어떤 학과 소속인지를 나타내는 데이터를 삽입할 수 없게 되므로 유의해야 한다. 따라서 반드시 다음과 같은 릴레이션 분해 원칙을 지켜서 분해해야 한다.

> 속성 A가 기본키인 릴레이션에서 A → C라는 함수적 종속이 존재하면서
> 이행적 함수적 종속(A → B ∧ B → C)이 성립할 때,
> 릴레이션을 {A, B}, {B, C}로 분해해서 이행적 함수적 종속을 제거한다.

[그림 10-24]의 학생지도와 교수소속 릴레이션은 모두 3NF다. 하지만 [그림 10-22]의 지도받다 릴레이션은 2NF이지만 3NF는 아니다. 이처럼 어떤 릴레이션이 2NF이지만 3NF가 아닐 때, 프로젝션을 통해서 의미상 동등한 2개의 3NF 릴레이션으로 분해할 수 있다. 반대로, 분해된 2개의 3NF 릴레이션을 자연 조인하면 아무런 정보 손실 없이 원래의 2NF 릴레이션으로 환원할 수 있다.



다만, 3NF 릴레이션은 2NF 릴레이션에 표현할 수 없는 정보를 표현할 수 있다. 예를 들면, 교수번호 '450'의 소속이 '전자'라는 사실을 지도받다 릴레이션에는 표현할 수 없지만 교수소속 릴레이션에는 표현할 수 있다.

이론적으로는 3NF에서도 이상 현상이 발생할 수 있지만 실무에서는 잘 발생하지 않는다. 따라서 실무에서의 효율성을 고려하여 3NF까지만 진행하는 것이 바람직하므로, 이 책에서도 3NF의 이상 현상에 대한 설명은 생략한다.

4 정규형 요약

비정규화된 릴레이션을 3NF까지 정규화하는 일련의 과정을 [그림 10-25]와 같이 요약하였다.

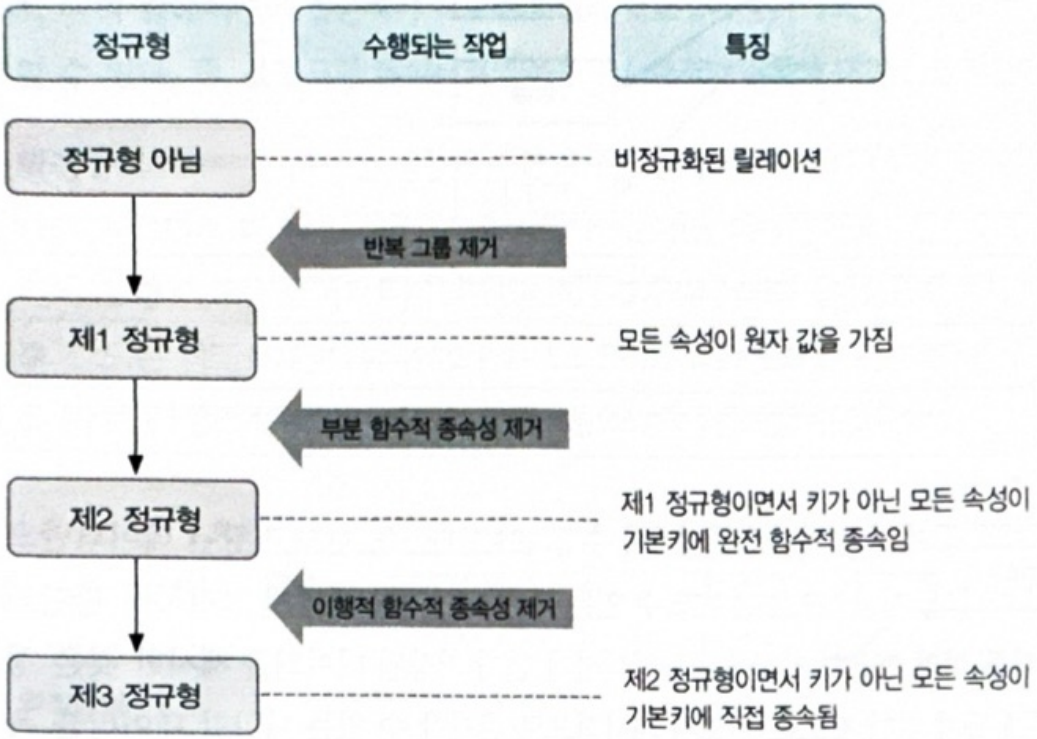
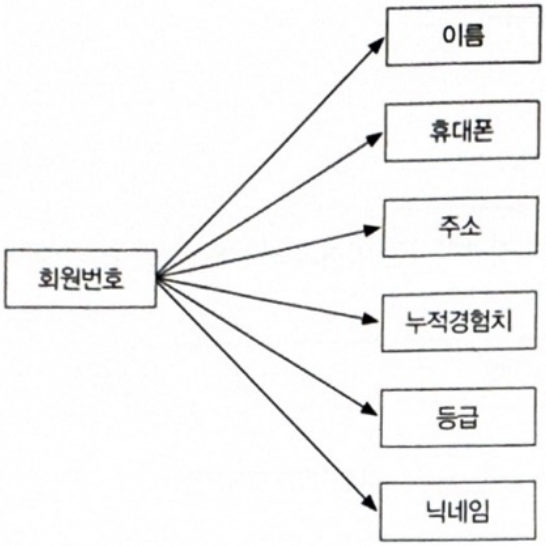


그림 10-25 정규화 수행 과정 및 정규형 특징 요약

이때 정규화 과정을 단계별로 모두 거쳐야 하는 것은 아니며, 릴레이션의 특징에 따라 어떤 정규형은 건너뛸 수 있다. 또한 기본키, 즉 결정자만으로 구성된 릴레이션이나 결정자와 단 하나의 종속자만으로 구성된 릴레이션 등은 굳이 정규화 과정이 필요하지 않다. 한편, 함수적 종속 관계를 분석하기 위해 FDD를 작성할 때 결정자 역할을 할 수 있는 대체키가 있으면 FDD가 너무 복잡해지므로, 대체키의 특징이 기본키와 동일한 경우 굳이 FDD에 대체키를 결정자로 표현하지 않아도 된다. 다만 [표 10-2]와 같이 별도로 대체키를 기록하는 것이 좋다.

표 10-2 회원 릴레이션의 정규화 내용

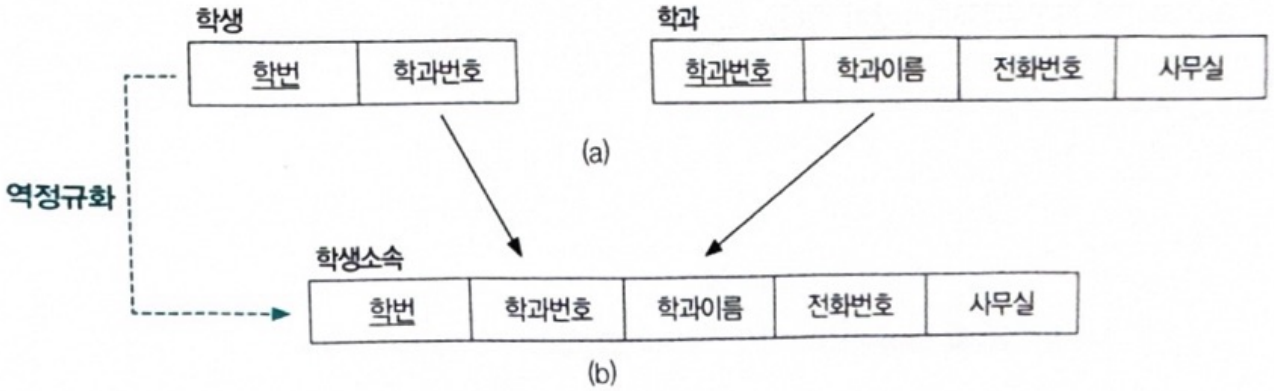
<table>
<tr><td>릴레이션 스키마</td><td colspan="2">회원(<u>회원번호</u>, 이름, 휴대폰, 주소, 누적경험치, 등급, 닉네임)</td></tr>
<tr><td>기본키</td><td colspan="2">회원번호</td></tr>
<tr><td>대체키</td><td colspan="2">휴대폰</td></tr>
<tr><td>함수적 종속</td><td colspan="2">회원번호 → (이름, 휴대폰, 주소, 누적경험치, 등급, 닉네임)</td></tr>
<tr><td>FDD</td><td colspan="2"> 
</td></tr>
<tr><td rowspan="3">정규형</td><td>1NF</td><td>모든 속성의 도메인이 원자 값이므로 1NF를 만족함</td></tr>
<tr><td>2NF</td><td>모든 속성이 기본키에 완전 함수적 종속이므로 2NF를 만족함</td></tr>
<tr><td>3NF</td><td>이행적 함수적 종속이 존재하지 않으므로 3NF를 만족함</td></tr>
</table>

정규화는 실제 데이터 값이 아닌 개념적 측면에서 수행해야 한다. 즉, 현재 저장된 데이터만으로 판단하는 것이 아니라, 개념적으로 존재할 수 있는 모든 데이터를 폭넓게 고려하여 판단해야 한다. 앞에서 다룬 여러 정규화 사례에서 릴레이션에 현재 저장된 데이터를 제시한 것은 정규화 과정의 이해를 돕기 위한 것으로, 반드시 개념적으로 존재할 수 있는 다양한 데이터를 폭넓게 고려해야 한다.

07 역정규화

정규화는 DB 설계의 중요한 요소로서, 정규화 과정이 진행될수록 데이터 중복이 감소하고 이상 현상도 감소하며, 데이터 무결성을 유지하는 데 필요한 응용 프로그램의 코드량이 감소한다는 장점이 있다. 하지만 성능 관점에서 보면 높은 정규형을 만족하는 릴레이션 스키마가 항상 최적의 상태라고 볼 수 없다. 한 정규형에서 다음 정규형으로 진행될 때마다 하나의 릴레이션이 최소 2개의 릴레이션으로 분해된다. 따라서 분해되기 전의 릴레이션을 대상으로 질의할 때는 조인 연산이 필요 없지만, 분해된 릴레이션을 대상으로 질의할 때는 같은 정보를 얻기 위해서 여러 릴레이션에 접근해야 하므로 조인의 필요성이 증가하고, 이는 심각한 성능 저하를 초래할 수 있다. 즉, 불필요한 데이터의 중복은 감소하지만 질의에 대한 응답시간은 증가할 수 있다.

역정규화denormalization는 성능에 대한 요구를 만족시키기 위해 데이터 중복과 이상 현상을 대가로 치르면서 더 낮은 정규형으로 되돌아가는 것을 의미한다. 즉, 빈번하게 수행되는 검색 질의의 수행 속도를 높이기 위해, 이미 분해된 2개 이상의 릴레이션을 합쳐서 하나의 릴레이션으로 만드는 작업이다. 예를 들어 [그림 10-26]과 같이 3NF를 만족하도록 분해된 두 릴레이션을 합쳐 '학생소속'이라는 하나의 릴레이션으로 역정규화할 수 있다.



(a) 제3 정규형을 만족하도록 분해된 학생과 학과 릴레이션
(b) 성능 향상을 위해 역정규화한 학생소속 릴레이션

그림 10-26 역정규화 예시

학생소속 릴레이션에서 학번이 '20248540'인 학생이 속한 학과의 이름과 전화번호를 검색하려면 다음과 같은 일반 검색 연산으로 가능하다.

```
SELECT   학과이름, 전화번호
FROM     학생소속
WHERE    학번 = '20248540';
```

하지만 정규화 과정을 통해 분리된 학생과 학과 릴레이션에서 동일한 정보를 검색하려면 다음과 같은 조인 연산으로 처리해야 한다. 조인 연산을 하면 일반 검색 연산을 할 때보다 검색 성능이 저하된다.

```
SELECT   학과이름, 전화번호
FROM     학생, 학과
WHERE    학번 = '20248540'
   AND   학생.학과번호 = 학과.학과번호
```

DB 설계자는 검색 성능을 고려해서 데이터 중복 및 이상 현상을 대가로 치르더라도 때로는 일부 릴레이션을 역정규화하여 응답시간을 단축시킬 필요가 있다. DB 응용은 대개 검색 질의의 비율이 다른 질의의 비율보다 훨씬 높다. 따라서 반드시 역정규화를 고려해야 하며, 역정규화를 수행한 경우 반드시 데이터 중복으로 인한 이상 현상을 관리해야 한다. 즉, 응용 프로그램을 이용해 별도로 이상 현상을 관리함으로써 데이터 무결성을 보장해야 한다. 한편, 사소한 성능 향상을 위해 데이터 무결성을 저해하는 것은 바람직하지 않으므로 심각한 성능 저하를 유발하는 경우에만 역정규화하는 것이 좋다.

08 정규화 사례

1 재능기부 릴레이션 정규화

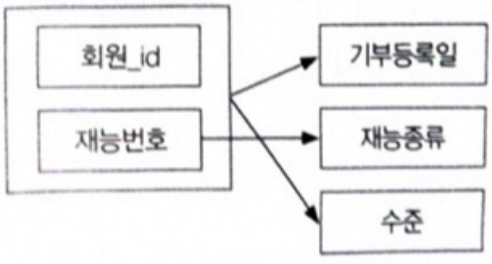
[그림 10-27]은 회원이 기부한 재능 정보를 표현하는 재능기부 릴레이션이다. 이를 정규화하는 과정은 [표 10-3]과 같이 요약할 수 있다.

재능기부

회원_id	재능번호	기부등록일	재능종류	수준
2331	1	2023/08/19	페인트	상
1254	2	2023/08/30	종이접기	중
0982	2	2023/10/13	종이접기	중
1105	3	2023/11/20	캘리그라피	중
2309	1	2024/01/09	페인트	상
0576	5	2024/02/10	목공	중
1892	4	2024/02/20	뜨개질	상

그림 10-27 재능기부 릴레이션

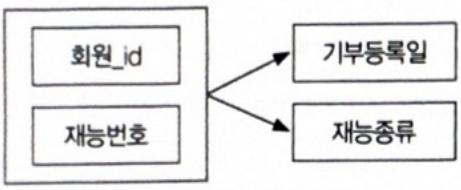
표 10-3 재능기부 릴레이션의 정규화 내용

릴레이션 스키마	재능기부(회원_id(FK), 재능번호, 기부등록일, 재능종류, 수준)
기본키	{회원_id, 재능번호}
대체키	없음
함수적 종속	{회원_id, 재능번호} → (기부등록일, 수준) /* 완전 함수적 종속만 표현함 */ 재능번호 → 재능종류
FDD	

정규형	1NF	모든 속성의 도메인이 원자 값이므로 1NF를 만족함
	2NF	부분 함수적 종속(재능번호 → 재능종류)이 존재하므로 2NF를 만족하지 않음 → **릴레이션 분해 필요함**
릴레이션 분해	재능기부(회원_id(FK), 재능번호(FK), 기부등록일, 수준) 재능정보(재능번호, 재능종류)	

[표 10-3]에서 확인할 수 있듯이 원래의 재능기부 릴레이션이 2개의 릴레이션으로 분해되었다. 따라서 분해된 재능기부 릴레이션은 다시 [표 10-4]와 같은 정규화 과정을 수행해야 한다. 한편, 분해된 재능정보 릴레이션은 앞에서 언급한 바와 같이 결정자와 단 하나의 종속자로 구성되어 있으므로 정규화할 필요가 없다.

표 10-4 분해된 재능기부 릴레이션의 정규화 내용

릴레이션 스키마	재능기부(회원_id(FK), 재능번호(FK), 기부등록일, 수준)	
기본키	{회원_id, 재능번호}	
대체키	없음	
함수적 종속	{회원_id, 재능번호} → {기부등록일, 수준}	
FDD		
정규형	1NF	모든 속성의 도메인이 원자 값이므로 1NF를 만족함
	2NF	모든 속성이 기본키에 완전 함수적 종속이므로 2NF를 만족함
	3NF	이행적 함수적 종속이 존재하지 않으므로 3NF를 만족함

2 고객 릴레이션 정규화

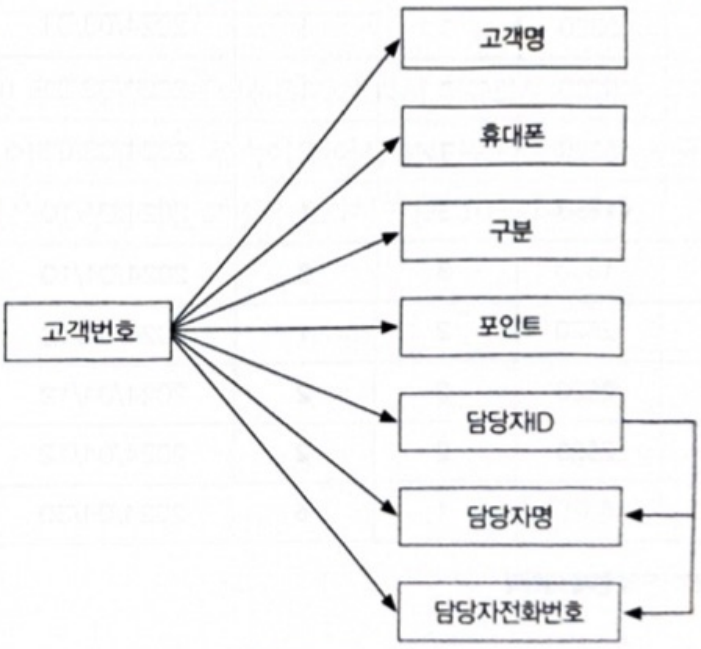
[그림 10-28]은 백화점 고객과 고객 관리 담당자 정보가 함께 저장된 고객 릴레이션이다. 이를 정규화하는 과정은 [표 10-5]와 같이 요약할 수 있다.

고객

고객번호	고객명	휴대폰	구분	포인트	담당자ID	담당자명	담당자전화번호
2501	김길동	010-3485-3949	VIP	20,100	100	장영민	041-575-1231
2230	박영숙	010-6383-5542	VIP	25,300	100	장영민	041-575-1231
2820	이창민	010-2683-5542	일반	12,000	200	민동진	041-575-1232
3240	이민수	010-4582-7630	일반	13,500	200	민동진	041-575-1232
3310	김영미	010-4585-7631	일반	8,000	200	민동진	041-575-1232
3215	박장수	010-2083-5302	VIP	32,000	100	장영민	041-575-1231
4511	이정이	010-3756-2957	일반	15,000	200	민동진	041-575-1232

그림 10-28 고객 릴레이션

표 10-5 고객 릴레이션의 정규화 내용

릴레이션 스키마	고객(고객번호, 고객명, 휴대폰, 구분, 포인트, 담당자ID, 담당자명, 담당자전화번호)
기본키	고객번호
대체키	휴대폰
함수적 종속	고객번호 → (고객명, 휴대폰, 구분, 포인트, 담당자ID, 담당자명, 담당자전화번호) 담당자ID → (담당자명, 담당자전화번호)
FDD	

정규형	1NF	모든 속성의 도메인이 원자 값이므로 1NF를 만족함
	2NF	모든 속성이 기본키에 완전 함수적 종속이므로 2NF를 만족함
	3NF	이행적 함수적 종속(고객번호 → 담당자ID, 담당자ID → (담당자명, 담당자전화번호))이 존재하므로 3NF를 만족하지 않음 **→ 릴레이션 분해 필요함**
릴레이션 분해	고객(고객번호, 고객명, 휴대폰, 구분, 포인트, **담당자ID(FK)**) 담당자(담당자ID, 담당자명, 담당자전화번호)	

분해된 고객 및 담당자 릴레이션을 각각 다시 정규화해야 하는데, 두 릴레이션 모두 부분 함수적 종속과 이행적 함수적 종속이 없어서 3NF를 만족하므로 나머지 정규화 과정은 생략한다.

3 주문목록 릴레이션 정규화

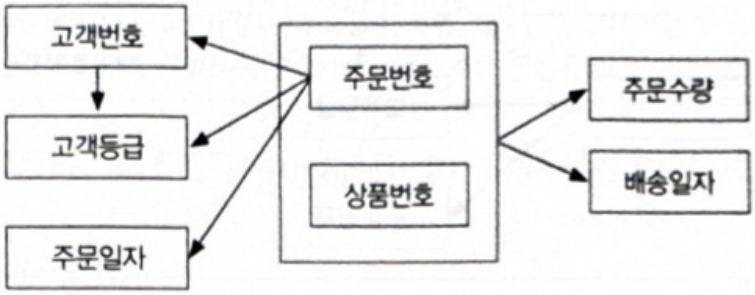
[그림 10-29]는 한 번에 하나 이상의 상품을 주문한 고객의 등급과 주문 수량 등을 저장하는 주문목록 릴레이션이다. 이를 정규화하는 과정은 [표 10-6]과 같이 요약할 수 있다.

주문목록

주문번호	상품번호	고객번호	고객등급	주문수량	주문일자	배송일자
1	110	5320	1	1	2024/03/01	2024/03/03
1	120	5320	1	1	2024/03/01	2024/03/03
1	250	5320	1	2	2024/03/01	2024/03/03
2	110	1350	3	1	2024/04/10	2024/04/11
2	220	1350	3	3	2024/04/10	2024/04/12
3	110	2520	2	1	2024/04/12	2024/04/13
3	250	2520	2	2	2024/04/12	2024/04/15
3	220	2520	2	2	2024/04/12	2024/04/15
4	120	5320	1	5	2024/04/30	2024/05/01

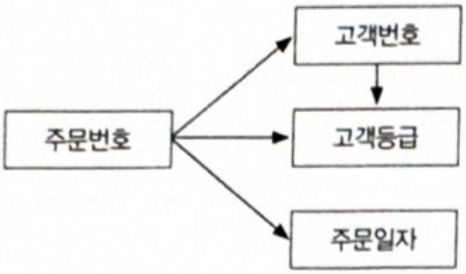
그림 10-29 주문목록 릴레이션의 현재 데이터

표 10-6 주문목록 릴레이션의 정규화 내용

<table>
<tr><td>릴레이션 스키마</td><td colspan="2">주문목록(주문번호, 상품번호(FK), 고객번호, 고객등급, 주문수량, 주문일자, 배송일자)</td></tr>
<tr><td>기본키</td><td colspan="2">{주문번호, 상품번호}</td></tr>
<tr><td>대체키</td><td colspan="2">없음</td></tr>
<tr><td>함수적 종속</td><td colspan="2">{주문번호, 상품번호} → (주문수량, 배송일자) /* 완전 함수적 종속만 표현함 */
주문번호 → (고객번호, 고객등급, 주문일자)
고객번호 → 고객등급</td></tr>
<tr><td>FDD</td><td colspan="2"> 
</td></tr>
<tr><td rowspan="2">정규형</td><td>1NF</td><td>모든 속성의 도메인이 원자 값이므로 1NF를 만족함</td></tr>
<tr><td>2NF</td><td>부분 함수적 종속(주문번호 → (고객번호, 고객등급, 주문일자))이 존재하므로
2NF를 만족하지 않음
→ 릴레이션 분해 필요함</td></tr>
<tr><td>릴레이션 분해</td><td colspan="2">주문목록(주문번호(FK), 상품번호(FK), 주문수량, 배송일자)
주문정보(주문번호, 고객번호, 고객등급, 주문일자)</td></tr>
</table>

분해된 주문목록 및 주문정보 릴레이션을 각각 다시 정규화해야 한다. 주문목록 릴레이션은 부분 함수적 종속과 이행적 함수적 종속이 없어서 3NF를 만족하므로 정규화 과정을 생략한다. 주문정보 릴레이션을 정규화하는 과정을 요약하면 [표 10-7]과 같다.

표 10-7 주문정보 릴레이션의 정규화 내용

<table>
<tr><td>릴레이션 스키마</td><td colspan="2">주문정보(<u>주문번호</u>, 고객번호, 고객등급, 주문일자)</td></tr>
<tr><td>기본키</td><td colspan="2">주문번호</td></tr>
<tr><td>대체키</td><td colspan="2">없음</td></tr>
<tr><td>함수적 종속</td><td colspan="2">주문번호 → (고객번호, 고객등급, 주문일자)
고객번호 → 고객등급</td></tr>
<tr><td>FDD</td><td colspan="2"> 
</td></tr>
<tr><td rowspan="3">정규형</td><td>1NF</td><td>모든 속성의 도메인이 원자 값이므로 1NF를 만족함</td></tr>
<tr><td>2NF</td><td>모든 속성이 기본키에 완전 함수적 종속이므로 2NF를 만족함</td></tr>
<tr><td>3NF</td><td>이행적 함수적 종속(주문번호 → 고객번호, 고객번호 → 고객등급)이 존재하므로 3NF를 만족하지 않음
→ 릴레이션 분해 필요함</td></tr>
<tr><td>릴레이션 분해</td><td colspan="2">주문정보(<u>주문번호</u>, 고객번호(FK), 주문일자)
고객분류(<u>고객번호</u>, 고객등급)</td></tr>
</table>

분해된 주문정보 및 고객분류 릴레이션도 각각 다시 정규화해야 하는데, 두 릴레이션 모두 부분 함수적 종속과 이행적 함수적 종속이 없으므로 3NF를 만족한다. 따라서 최초의 주문목록 릴레이션은 정규화 과정을 통해 다음과 같은 3개의 릴레이션으로 분해된다.

주문목록(<u>주문번호(FK), 상품번호(FK)</u>, 주문수량, 배송일자)
주문정보(<u>주문번호</u>, 고객번호(FK), 주문일자)
고객분류(<u>고객번호</u>, 고객등급)

4 온라인 쇼핑몰 DB 구축을 위한 정규화

9장 6.1절에서 소개한 온라인 쇼핑몰 DB 구축을 위한 논리적 스키마 모델링 사례에서 다음과 같은 최종 릴레이션 스키마가 정의되었다.

```
상품(상품번호, 구분, 상품명, 사이즈, 판매가격, 구매가격)
고객(id, pw, 이름, 휴대폰, 주소, 포인트)
고객배송지(고객 id(FK), 순서, 배송지)
구매처(구매처번호, 상호, 연락처, 주소, 대표자, 사업자등록번호, 주요취급품목)
판매하다(판매번호, 상품번호(FK), 고객id(FK), 판매일시, 수량, 배송일자, 결제금액)
구입하다(구매번호, 상품번호(FK), 구입일자, 구매처번호(FK), 수량, 구입금액)
```

각 릴레이션을 정규화하는 과정은 [표 10-8]~[표 10-13]과 같이 요약할 수 있다.

표 10-8 상품 릴레이션의 정규화 내용

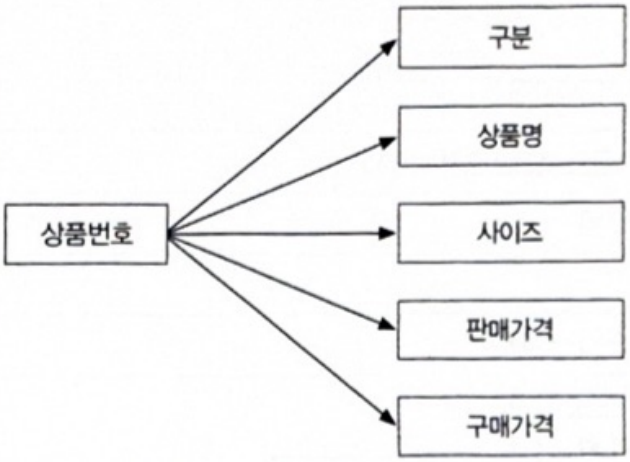
릴레이션 스키마	상품(상품번호, 구분, 상품명, 사이즈, 판매가격, 구매가격)	
기본키	상품번호	
대체키	없음	
함수적 종속	상품번호 → (구분, 상품명, 사이즈, 판매가격, 구매가격)	
FDD		
정규형	1NF	모든 속성의 도메인이 원자 값이므로 1NF를 만족함
	2NF	모든 속성이 기본키에 완전 함수적 종속이므로 2NF를 만족함
	3NF	이행적 함수적 종속이 존재하지 않으므로 3NF를 만족함

표 10-9 고객 릴레이션의 정규화 내용

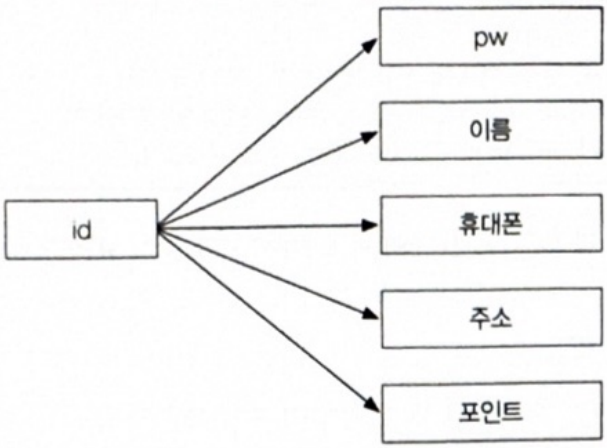
릴레이션 스키마	고객(id, pw, 이름, 휴대폰, 주소, 포인트)	
기본키	id	
대체키	휴대폰	
함수적 종속	id → (pw, 이름, 휴대폰, 주소, 포인트)	
FDD		
정규형	1NF	모든 속성의 도메인이 원자 값이므로 1NF를 만족함
	2NF	모든 속성이 기본키에 완전 함수적 종속이므로 2NF를 만족함
	3NF	이행적 함수적 종속이 존재하지 않으므로 3NF를 만족함

표 10-10 고객배송지 릴레이션의 정규화 내용

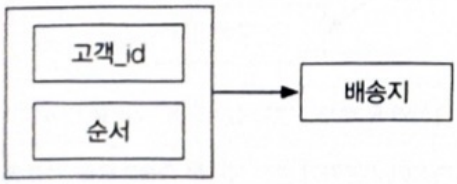
릴레이션 스키마	고객배송지(고객_id(FK), 순서, 배송지)	
기본키	{고객_id, 순서}	
대체키	없음	
함수적 종속	{고객_id, 순서} → 배송지	
FDD		
정규형	1NF	모든 속성의 도메인이 원자 값이므로 1NF를 만족함
	2NF	모든 속성이 기본키에 완전 함수적 종속이므로 2NF를 만족함
	3NF	이행적 함수적 종속이 존재하지 않으므로 3NF를 만족함

표 10-11 구매처 릴레이션의 정규화 내용

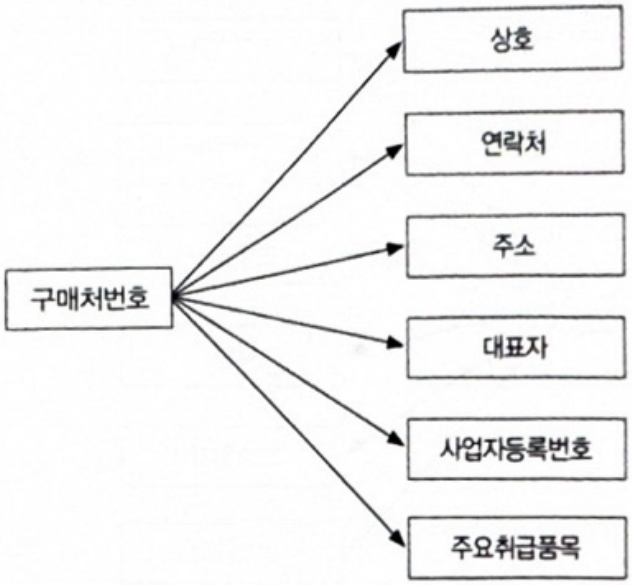
<table>
<tr><td>릴레이션 스키마</td><td colspan="2">구매처(<u>구매처번호</u>, 상호, 연락처, 주소, 대표자, 사업자등록번호, 주요취급품목)</td></tr>
<tr><td>기본키</td><td colspan="2">구매처번호</td></tr>
<tr><td>대체키</td><td colspan="2">사업자등록번호</td></tr>
<tr><td>함수적 종속</td><td colspan="2">구매처번호 → (상호, 연락처, 주소, 대표자, 사업자등록번호, 주요취급품목)</td></tr>
<tr><td>FDD</td><td colspan="2"> 
</td></tr>
<tr><td rowspan="3">정규형</td><td>1NF</td><td>모든 속성의 도메인이 원자 값이므로 1NF를 만족함</td></tr>
<tr><td>2NF</td><td>모든 속성이 기본키에 완전 함수적 종속이므로 2NF를 만족함</td></tr>
<tr><td>3NF</td><td>이행적 함수적 종속이 존재하지 않으므로 3NF를 만족함</td></tr>
</table>

표 10-12 판매하다 릴레이션의 정규화 내용

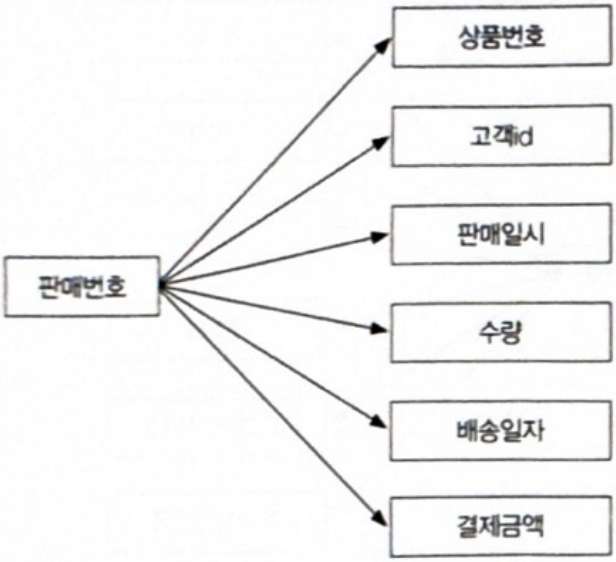
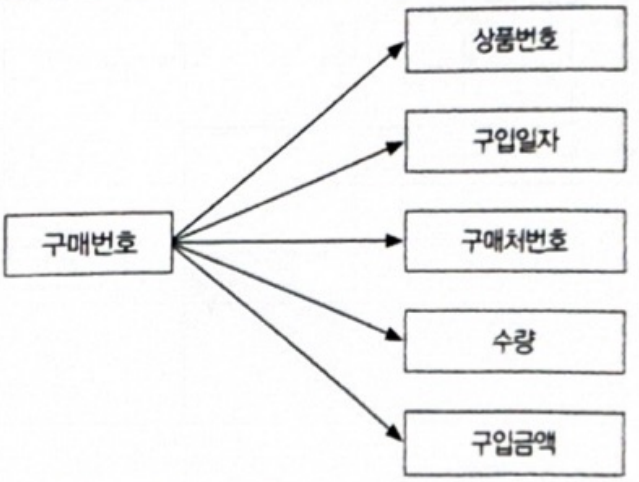
릴레이션 스키마	판매하다(판매번호, 상품번호(FK), 고객id(FK), 판매일시, 수량, 배송일자, 결제금액)	
기본키	판매번호	
대체키	{상품번호, 고객id, 판매일시}	
함수적 종속	판매번호 → (상품번호, 고객id, 판매일시, 수량, 배송일자, 결제금액)	
FDD		
정규형	1NF	모든 속성의 도메인이 원자 값이므로 1NF를 만족함
	2NF	모든 속성이 기본키에 완전 함수적 종속이므로 2NF를 만족함
	3NF	이행적 함수적 종속이 존재하지 않으므로 3NF를 만족함

표 10-13 구입하다 릴레이션의 정규화 내용

릴레이션 스키마	구입하다(구매번호, 상품번호(FK), 구입일자, 구매처번호(FK), 수량, 구입금액)	
기본키	구매번호	
대체키	(상품번호, 구입일자, 구매처번호)	
함수적 종속	구매번호 → (상품번호, 구입일자, 구매처번호, 수량, 구입금액)	
FDD		
정규형	1NF	모든 속성의 도메인이 원자 값이므로 1NF를 만족함
	2NF	모든 속성이 기본키에 완전 함수적 종속이므로 2NF를 만족함
	3NF	이행적 함수적 종속이 존재하지 않으므로 3NF를 만족함

이 사례처럼 논리적 스키마 모델링 단계를 원칙대로 잘 수행하면 정규화 과정을 거쳐도 릴레이션 스키마가 크게 변하지 않는다. 9장 6.2절에서 소개한 문화센터 DB 구축을 위한 최종 릴레이션 스키마 역시 정규화 과정을 거쳐도 릴레이션 스키마에 변화가 없으므로 자세한 설명은 생략한다.